DEPARTMENT OF INFORMATION TECHNOLOGY

INDIAN INSTITUTE OF ENGINEERING SCIENCE AND TECHNOLOGY, SHIBPUR

MAY 2025

Food Classification Using Convolutional Neural Network

A Report

on

B. Tech Third Semester Mini Project -I (IT2291)

Presented BY:

- Tarun Srivastava, 2023ITB026
- Aparup Ghosh, 2023ITB044
- Pooja Yadav, 2023ITB103

Date of Presentations: 09- May- 2025

Introduction

Image classification is a fundamental task in computer vision, where input images are analysed and assigned labels from predefined categories. This project focuses on **food image classification** using **Convolutional Neural Networks (CNNs)**, aiming to automatically identify dishes such as:

Tacos

breakfast burrito

These systems have practical applications in food logging, nutritional tracking, smart restaurant systems, and cultural documentation.

The motivation behind this project comes from the rising need for automated food recognition in diet apps and delivery platforms. Food images are visually complex due to variations in colour, texture, shape, presentation, and lighting, making deep learning an ideal solution.

CNNs are highly effective for food classification as they automatically learn visual features like colour, texture, and shape, while handling spatial variability and generalizing well across diverse food presentations. Their success in large-scale benchmarks like ImageNet has led to real-world adoption in applications such as CalorieMama and FoodAI. Research using datasets like Food-101, and models like AlexNet, VGG16, and GoogleNet, has shown strong performance. Studies by Kawano and Yanai (2014) and Ciocca et al. (2017) further support CNNs' reliability in recognizing a wide range of cuisines.

For this project, we used the **Food-101** dataset for both training and testing purposes. Food-101 offers a diverse range of labelled food images, making it a suitable benchmark for evaluating model performance in real-world classification scenarios.

This project aims to design and evaluate a CNN-based food classifier, explore transfer learning strategies, and apply deep learning techniques to a real-world visual recognition problem.

Fundamentals and Theory

Artificial Neural Networks (ANNs) are computing models inspired by the human brain. They consist of interconnected processing elements called **neurons** that work together forming network called as **neural network** to solve problems like pattern recognition or classification by learning from examples. Learning in ANNs involves adjusting the connections (weights) between neurons to minimize errors.

❖ Artificial Neural Network Model

Neuron: A neuron is the fundamental unit of a neural network. It receives input, applies weights and bias, passes the result through an activation function, and produces an output.

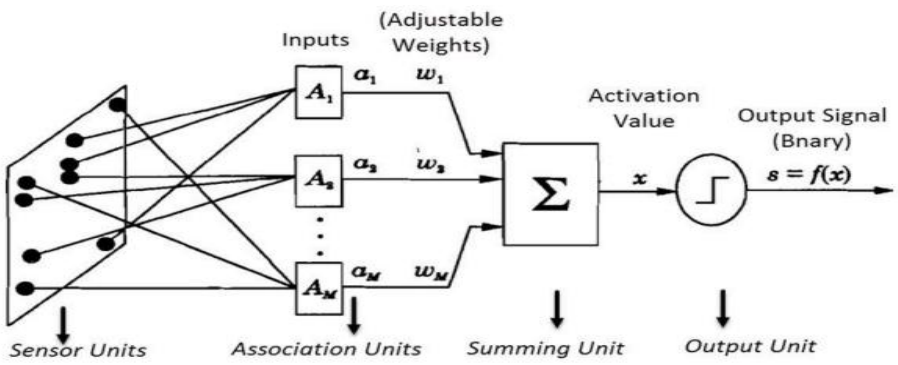
An artificial neuron models a biological neuron. It includes:

Input connections from other neurons, Weights and threshold (bias) to process inputs, Activation function (e.g., sigmoid, ReLU) to produce output.

Perceptron: A perceptron is the simplest type of artificial neural network unit that makes decisions by weighing input signals, adding a bias, and passing the result through an activation function (typically a step function).

I. Single Layer Perceptron

It is the earliest form of neural network. A single-layer perceptron computes a weighted sum of binary inputs, adds a bias, and outputs 0 or 1 using a step function.



Mathematically, the output of a perceptron is defined as:

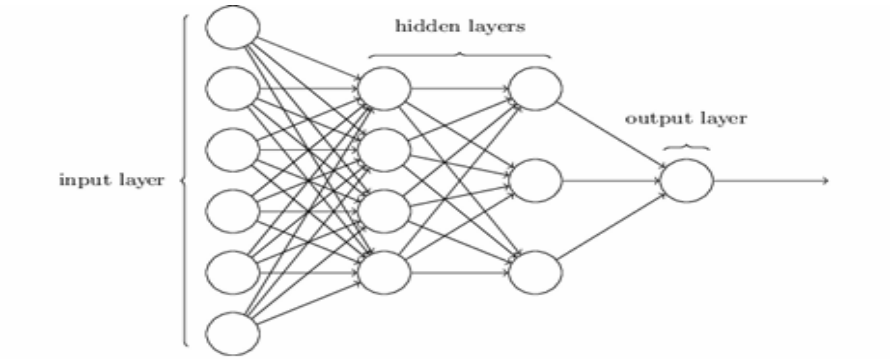
$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases} \qquad \text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

,

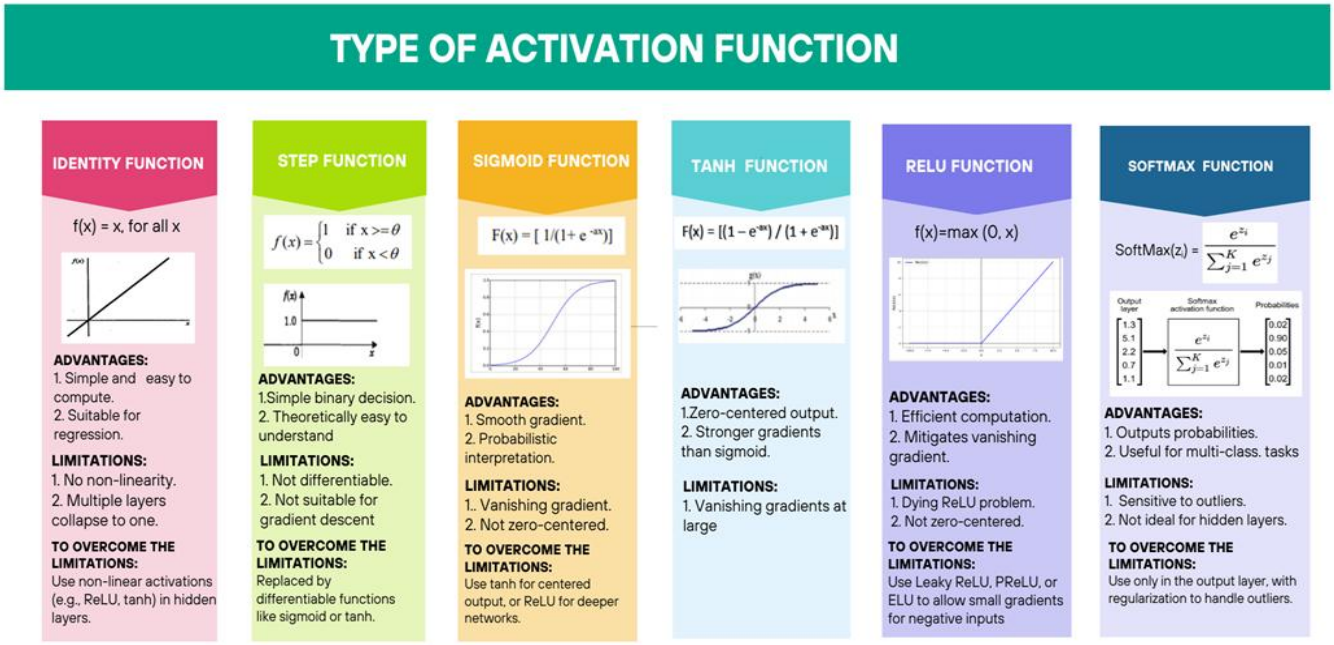
It serves as a **linear classifier** that divides input space with a hyperplane.

II. Multi-Layer Perceptron Model

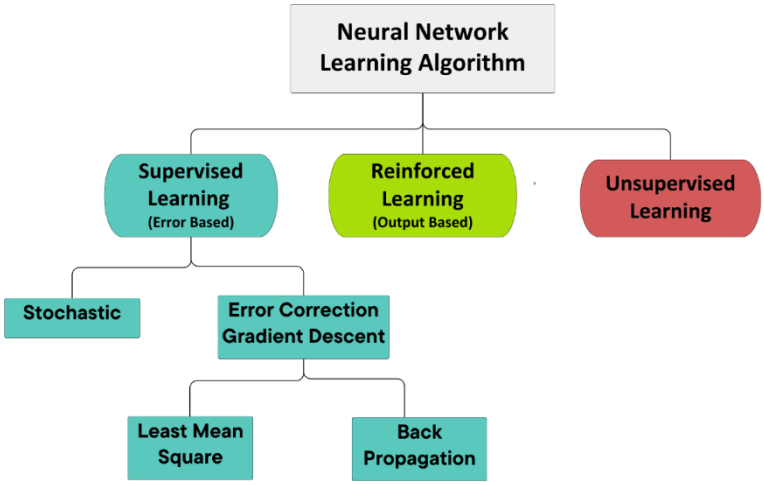
This model includes one or more **hidden layers** between input and output layers, allowing it to handle complex, non-linear problems using non-linear activation functions like sigmoid or tanh.



Multi-Layer Perceptron



❖ Different Learning Algorithms:



Cost Function

The cost function, or loss function, measures the difference between the predicted output and the true label. A commonly used function is the quadratic (mean squared error) cost, defined as:

$$C = \frac{1}{2n} \sum_{x} \|y(x) - a^{L}(x)\|^2,$$

Where, **n** is the number of training examples, **x** is an input from the training set, **y(x)** is the desired output (label) for input x, **a(x)** is the actual output from the neural network for input x, **w** and **b** are the weights and biases of the network

> **Purpose :** The goal of training a neural network is to **minimize the cost function**, which corresponds to improving the accuracy of predictions. This is achieved using optimization techniques like **gradient descent**.

Gradient Descent

Gradient descent is an optimization algorithm used to minimize the cost function by iteratively adjusting weights and biases. The updates are computed as:

$$w \to w - \eta \frac{\partial C_0}{\partial w} - \frac{\eta \lambda}{n} w \quad , \quad b \to b - \eta \frac{\partial C_0}{\partial b}$$

Where, **η** is the learning rate. Gradients point in the direction of maximum increase, so the algorithm moves in the opposite direction to reduce the cost.

- **Stochastic Gradient Descent (SGD)**

SGD is a faster variant of gradient descent, updating weights using a single or mini batch of examples rather than the full dataset.

Advantages : Faster updates, better generalization due to noise, escapes local minima more effectively.

Backpropagation

Backpropagation is a fundamental algorithm used to train feedforward neural networks by efficiently computing the gradient of the cost function with respect to each weight and bias in the network. It consists of:

- Forward pass: Compute activations and cost
- Backward pass: Compute error terms and gradients

Key equations:

Summary: the equations of backpropagation

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \qquad \text{(BP1)}$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \qquad \text{(BP2)}$$

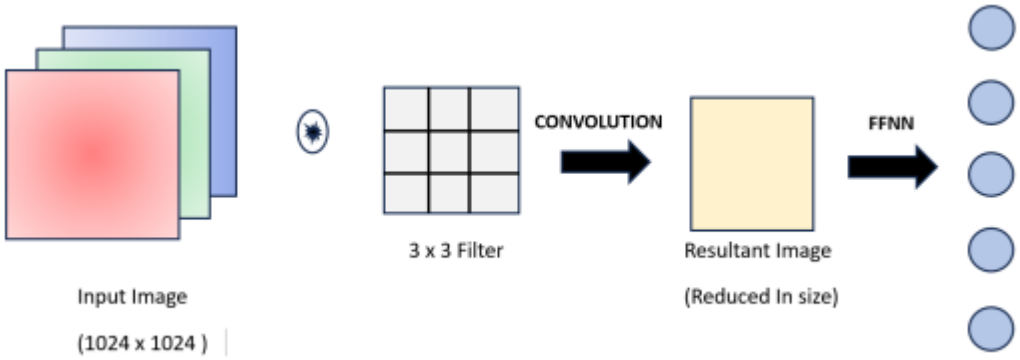
$$\frac{\partial C}{\partial b^l_j} = \delta^l_j \qquad \text{(BP3)}$$

$$\frac{\partial C}{\partial w^l_{jk}} = a^{l-1}_k \delta^l_j \qquad \text{(BP4)}$$

This allows efficient parameter updates in conjunction with gradient descent.

Convolutional Neural Networks (CNNs)

CNNs are widely used in image processing tasks like classification and object detection due to their ability to process high-dimensional input data efficiently. For example, a 1024×1024 RGB image contains over 3 million values, making traditional fully connected networks impractical. CNNs reduce input size while preserving essential spatial features, making them ideal for such tasks. In RGB images, convolution is applied across all three channels. Filters extract features like textures and shapes from the combined colour data, resulting in enhanced representation for classification.



Basic Convolutional Neural Network

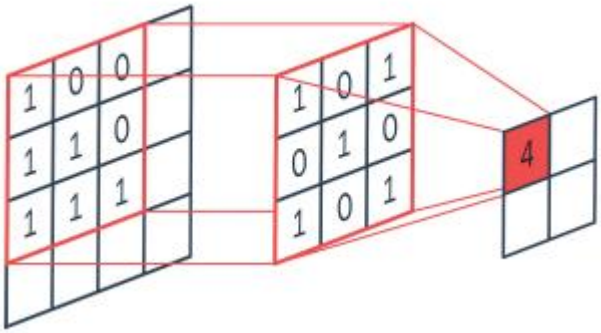
Convolutional Operation

The convolution operation between two functions f and g (denoted by $f * g$) is defined as:

$$f(x) * g(x) = \int_{-\infty}^{\infty} f(t)g(x-t)dt$$

Convolutional Operation

Convolution involves sliding a filter (kernel) over the input image, performing element-wise multiplication, and summing the results to produce a **feature map**. Mathematically, if the image is of size $(n \times n)$ and the filter is $(f \times f)$, the output size is $(n - f + 1) \times (n - f + 1)$.



Matrix Convolution

Edge Detection Using Convolution

CNN filters help detect edges—key features in images.

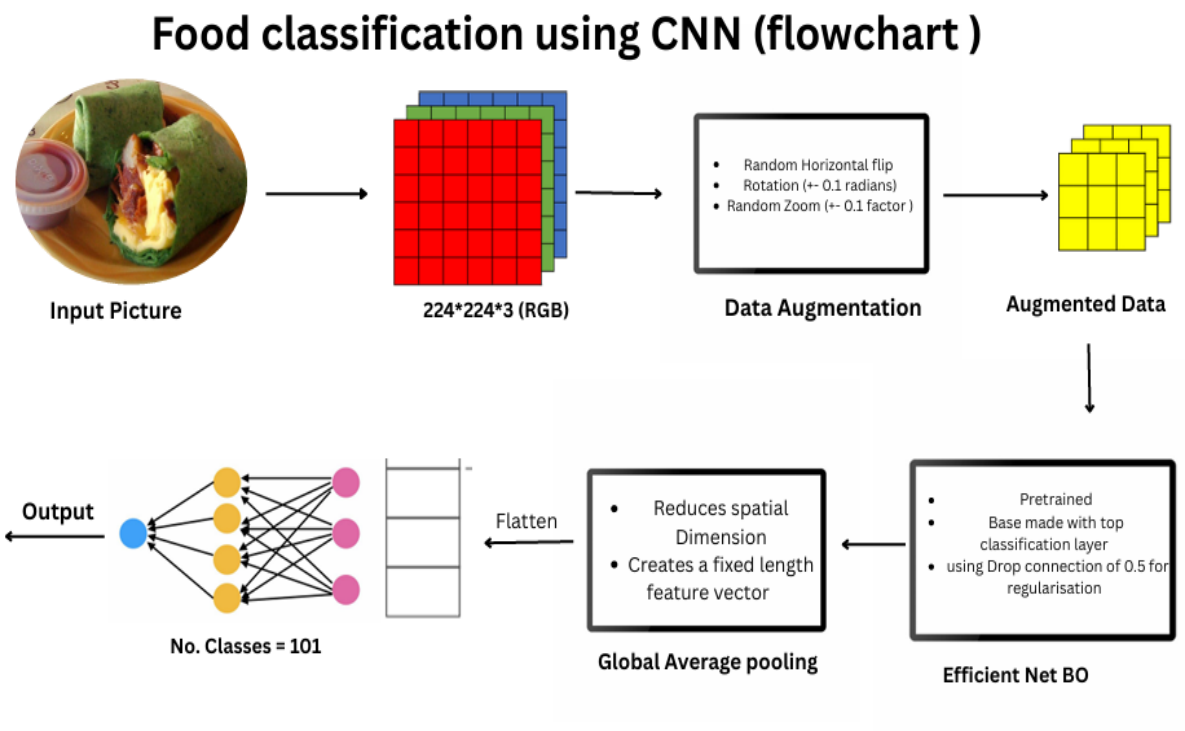
- **Vertical Edge Detection:** Uses kernels to highlight changes along the vertical line (horizontal axis). A commonly used kernel is:

$$\text{Vertical Edge Detector} - (\text{g}) = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

- **Horizontal Edge Detection:** Captures variations along the horizontal line (vertical axis). A typical kernel is:

$$\text{Horizontal Edge Detector} - (\text{g'}) = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Architecture of Model



The following describes each component of the food classification model based on EfficientNetB0, as visualized in the flowchart:

1. Input Picture

The model takes an input image from the Food-101 dataset. These images represent various food items and are typically in JPEG format with varying resolutions and lighting conditions.

2. Image Resizing (224 × 224 × 3 RGB)

All input images are resized to a fixed dimension of **224 × 224 pixels** with **3 colour channels** (RGB). This standardization ensures compatibility with the EfficientNetB0 model, which expects inputs of this size.

3. Data Augmentation

To improve generalization and prevent overfitting, random data augmentation techniques are applied during training:

- **Random horizontal flipping**
- **Random rotation** within ±0.1 radians
- **Random zooming** within a ±10% range

These transformations simulate real-world image variations and enhance the model's robustness.

4. EfficientNetB0 (Feature Extraction)

The core of the model uses a **pretrained EfficientNetB0** architecture. This convolutional neural network is known for its efficiency and performance. It is used without the top classification layer (include_top=False) and acts purely as a **feature extractor**. Additionally, a **DropConnect** regularization rate of **0.5** is used to reduce overfitting.

5. Global Average Pooling

The feature maps output by EfficientNetB0 are passed through a Global Average Pooling layer, which reduces each feature map to a single value. This converts the multi-dimensional output into a 1D feature vector, preserving the most important information while reducing dimensionality.

6. Dense Layer (Classification Head)

A fully connected **Dense layer** is added at the end with **101 output neurons**, corresponding to the 101 food categories in the dataset. A **Softmax activation function** is used to produce a probability distribution over the classes.

7. Output

The final output of the model is the **predicted food class**, which has the highest probability from the Softmax layer. This corresponds to one of the 101 predefined labels in the Food-101 dataset.

Tools Used in Food Classification Using CNNs

1. Padding

Padding adds zero layers around the input to prevent loss of edge information. With filter size *f* and input size *n*, padding *p* ensures output size match input:

$$2p = f - 1 \; p = (f - 1) \; 2 \text{ , where f is odd}$$

2. Striding

Stride determines the number of pixels the filter moves per step. A larger stride reduces the output size and computation, acting as a form of downsampling. If the grid size is n1 × n2 and the filter is square grid with size f and the stride is s then the resultant grid will be of size

$$(\lfloor (n1 - f) \; s + 1 \rfloor) \times (\lfloor (n2 - f) \; s + 1 \rfloor)$$

3. Pooling Layer

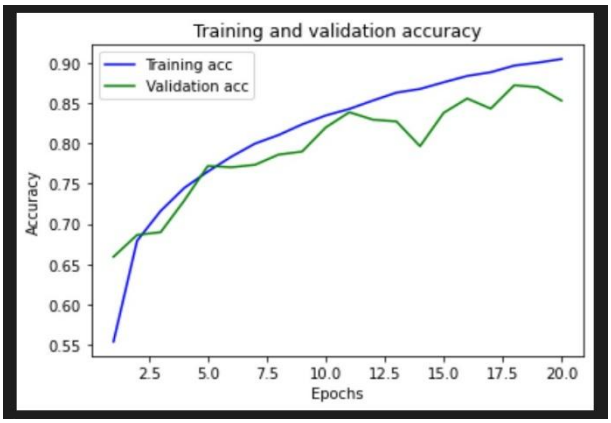
Global Average Pooling is a layer used in CNNs to reduce the spatial dimensions of feature maps by averaging each feature map into a single value. Unlike fully connected layers, it dramatically reduces the number of parameters, helps prevent overfitting, and retains important spatial information. It's widely used in efficient models like EfficientNet for lightweight and fast classification.

Result and Evaluation

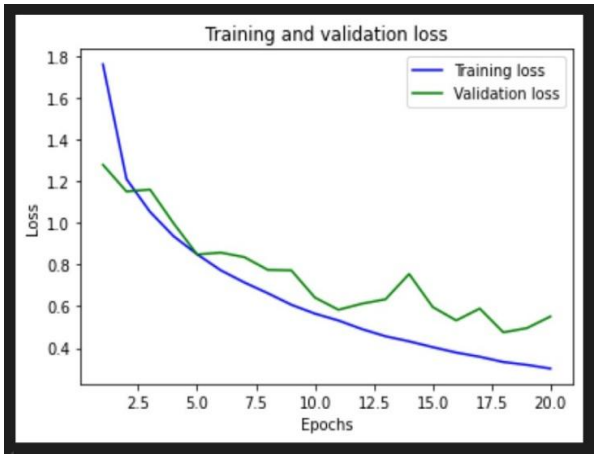
Accuracy evaluates model performance as the proportion of correct predictions. The model achieved a training accuracy of 90%, indicating strong learning on the training data. It also reached a validation accuracy of 85.75%, demonstrating good generalization to unseen data.

Prediction is the process where the network outputs probabilities for each class, and the class with the highest probability is selected as the final predicted label.

Our model loss and accuracy curves :



Epochs v/s Accuracy



Epochs v/s Loss

Conclusion

This project successfully demonstrated the effectiveness of Convolutional Neural Networks (CNNs) for food image classification using the Food-101 dataset. By leveraging deep learning techniques and transfer learning with the EfficientNetB0 model, we developed a robust classifier that achieved **90% accuracy on the training set** and **85.75% accuracy on the validation set**. These results highlight the power of CNNs in capturing the visual complexity of food images, offering valuable insights for real-world applications such as diet tracking, food recognition, and personalized nutrition systems.

Future work can further enhance the model by exploring more advanced architectures and larger datasets to improve generalization.

References

- Neural Network and Deep Learning, By Michael Nielsen
- Handbook of Medical Image Computing and Computer Assisted Intervention, By Jonas Teuwen, Nikita Moriakov
- https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/
- https://doi.org/10.1016/j.cviu.2018.08.002

Appendix

This section presents the model architecture used for **food classification using convolutional neural networks (CNNs) on the Food-101 dataset**. The model utilizes a pre-trained EfficientNetB0 backbone with data augmentation techniques to reduce overfitting.

CODE:

CIFAR-10 Image Classification using CNN ::

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# Build CNN Model
  cnn = models.Sequential([
    # Convolutional Layer 1
    layers.Conv2D(32, (3, 3), padding='same', activation='relu',
    input_shape=(32, 32, 3)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Dropout(0.25),
    # Convolutional Layer 2
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
```

```
layers.MaxPooling2D(pool_size=(2, 2)),
layers.Dropout(0.25),
# Convolutional Layer 3
layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
layers.BatchNormalization(),
layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D(pool_size=(2, 2)),
layers.Dropout(0.25),
# Fully Connected Layers
layers.Flatten(),
layers.Dense(512, activation='relu'),
layers.BatchNormalization(),
layers.Dropout(0.5),
layers.Dense(10, activation='softmax')  # 10 output classes
])
# Compile the model
  cnn.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='sparse_categorical_crossentropy',
         metrics=['accuracy'])
# Data Augmentation
 datagen = ImageDataGenerator(
rotation_range=15,
width_shift_range=0.1,
height_shift_range=0.1,
horizontal_flip=True
)
```

```
# Assuming X_train and y_train are loaded
datagen.fit(X_train)
# Train the model
cnn.fit(datagen.flow(X_train, y_train, batch_size=64), epochs=50)
```

Food-101 Classification using EfficientNetB0 ::

```
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
# Load Food-101 dataset
train_data = tf.keras.preprocessing.image_dataset_from_directory(
    '../input/food41/images',
    validation_split=0.2,
    subset='training',
    seed=123,
    image_size=(224, 224),
    batch_size=32,
    label_mode='categorical'
)
valid_data = tf.keras.preprocessing.image_dataset_from_directory(
    '../input/food41/images',
    validation_split=0.2,
    subset='validation',
    seed=123,
    image_size=(224, 224),
    batch_size=32,
    label_mode='categorical'
```

```
)
# Data Augmentation
data_augmentation = tf.keras.Sequential([
   layers.experimental.preprocessing.RandomFlip("horizontal", input_shape=(224, 224, 3)),
   layers.experimental.preprocessing.RandomRotation(0.1),
   layers.experimental.preprocessing.RandomZoom(0.1),
])
# Build EfficientNetB0 Model
model = Sequential([
   data_augmentation,
   tf.keras.applications.EfficientNetB0(
      input_shape=(224, 224, 3),
      weights='imagenet',
      include_top=False,
      drop_connect_rate=0.5
   ),
   layers.GlobalAveragePooling2D(),
   layers.Dense(101, activation='softmax')  # 101 classes in Food-101
])
# Compile the model
model.compile(optimizer='adam',
          loss='categorical_crossentropy',
          metrics=['accuracy'])
# Train the model
model.fit(train_data, validation_data=valid_data, epochs=20)
```